Lab 3: Process Credentials, PAM and the Authentication Stack

Computing Systems Security, 2026-27 MCS and MEI, University of Minho

Vítor Francisco Fonte vff@di.uminho.pt

Every access-control decision on a Unix system rests on one prior question: *who is this process?* This lab starts with process credentials, the UIDs a process carries, then opens up the machinery Linux uses to answer ”who are you?” at login: the Name Service Switch (NSS), which resolves accounts, and **PAM** (Pluggable Authentication Modules), the stack that actually authenticates. You will trace a real authentication, deliberately break it and repair it, and then harden it with account lockout and password-quality policy. The block this lab opens is **authentication and authorisation**; the next two labs build federated identity (Lab 4) and SSH certificates (Lab 5) on top of it.

This lab edits authentication configuration. You can lock yourself out. Two rules, non-negotiable: **(1) snapshot before you start**, and **(2) keep one root shell open at all times** (`sudo -i` in a dedicated terminal) so you can always undo a change. Every destructive step below tells you how to recover.

Contents

- Learning Objectives
- Before the lab (at home, ~30 min)
- Part A: Who is a process? Credentials and NSS
- Part B: The PAM stack
- Part C: Break it, then repair it
- Part D: Hardening the stack
- If it does not work
- Reflection
- References
- Deliverables

Learning Objectives

By completing this lab, students will be able to:

1. Explain real, effective and saved UIDs, and how a process's identity is established (the full setuid *transitions* are Block 3).
2. Describe how NSS resolves an account and where account and credential data live (`/etc/passwd`, `/etc/shadow`, `nsswitch.conf`).
3. Read a PAM configuration: the four module types, the control flags, and how a stack composes.
4. Diagnose and repair a broken authentication chain.
5. Apply account-lockout (`pam_faillock`) and password-quality (`pam_pwquality`) policy, and observe their effect.

What from this lab is assessed. Two things, both handed in with **Lab report 2** at the end of the block (Lab 5): your answers to the preparation questions **P1 and P2**, and your **Reflection**. The exercises themselves are not handed in. They are the practice that Lab 5 and its report build on, so do them properly and keep your notes: Lab 5 assumes you can read a PAM stack.

Before the lab (at home, ~30 min)

Do the setup and reading below *before* the session, so the 2 hours go on reading and breaking the PAM stack, not on setup. Boot the course VM. **Take a snapshot named `lab3-clean` now** (UTM snapshot control / VirtualBox *Machine → Take Snapshot…*).

Open **two** terminals inside the VM:

- **Terminal R (the safety net):** run `sudo -i` and leave it as root. Do not close it for the duration of the lab. If a later step locks you out of new logins, you repair the damage from here.
- **Terminal W (the working shell):** your ordinary `student` shell, where you run the exercises.

```
# in Terminal W
mkdir -p ~/ssc-lab3 && cd ~/ssc-lab3
```

Preparation questions (submit with Lab report 2)

- **P1.** Name the four PAM module *types* and, in a few words, what each governs.
- **P2.** On Ubuntu, which shared file does `/etc/pam.d/sudo` pull its auth rules from?

Part A: Who is a process? Credentials and NSS

Exercise 1: Process credentials.

```
id                      # your real/effective UID and GID, and groups
cat /proc/self/status | grep -E '^(Uid|Gid):'   # the Uid: and Gid: lines
```

1a. The `Uid:` line shows four numbers. Name each one, and say in one sentence what it is for.

Exercise 2: Where does an account come from? NSS.

The Name Service Switch decides *which sources* answer ”does user X exist?”. Its policy is `/etc/nsswitch.conf`.

```
grep -E '^(passwd|group|shadow):' /etc/nsswitch.conf
getent passwd student          # resolve 'student' through NSS
getent passwd 0                # resolve uid 0
```

2a. Which sources does `nsswitch.conf` consult for `passwd`? Record the list your machine shows. Then suppose the machine also looked accounts up in a **directory on another machine**: say what that would mean for the trust boundary of authentication.

Two names you will meet. LDAP is a protocol for such a directory of accounts; **SSSD** is the local service through which a Linux machine queries one (in `nsswitch.conf` it appears as the source `sss`). You do not need either installed: the question is what changes when ”does this user exist?” is answered by another machine.

Exercise 3: Where do credentials live?

```
getent passwd svc-report                # the service account from Lab 1
sudo getent shadow student | cut -d: -f1,2   # the hash field (field 2)
```

3a. `/etc/passwd` is world-readable but `/etc/shadow` is not. Why is one public and the other not?

3b. What attack does keeping the hashes out of `/etc/passwd` frustrate?

Part B: The PAM stack

PAM lets each service (login, `sshd`, `sudo`, `su`) compose authentication from stackable modules, configured in `/etc/pam.d/`. There are four **module types**:

- **auth**: prove identity (a password, a key, a token);
- **account**: is the account allowed to log in now (expiry, time restrictions)?
- **password**: govern credential *changes* (quality rules);
- **session**: set up/tear down the session (mounts, limits, logging).

Each line also carries a **control flag** (`required`, `requisite`, `sufficient`, `optional`, or the richer `[success=... default=...]` form) that decides how the stack composes.

Exercise 4: Read a real stack.

```
cat /etc/pam.d/sudo
cat /etc/pam.d/common-auth        # the shared auth stack most services include
```

4a. In `/etc/pam.d/sudo`, identify which lines are `@included` from the `common-*` files.

4b. What does `pam_unix.so` do in the `auth` stack, and where does it read the hash it checks against? Tie this back to Part A.

4c. Explain the difference between the `required` and `sufficient` control flags.

4d. Ubuntu's generated `common-auth` does **not** use the literal word `sufficient`: it uses the richer `[success=... default=...]` form. Read the `pam_unix.so` line and say what its `[success=... default=ignore]` action means in stack terms. Then contrast it with a hypothetical `auth sufficient pam_unix.so`: how would the stack behave differently?

Part C: Break it, then repair it

You will now break the authentication stack for **sudo only** (chosen deliberately: breaking `sudo` cannot lock you out of *login*, and Terminal R is already root). Snapshot integrity plus Terminal R are your safety net.

⚠ **Breaking sudo will not stop you logging in, but it can stop you repairing the machine.** The edit touches `/etc/pam.d/sudo` only, so `login` and `sshd` keep working and you will still be able to sign in as `student`. But **your only ordinary route to root is sudo, and that is what you are about to break**, and the backup you make in Exercise 5 goes to `/root`, which you cannot read without root. So if **Terminal R closes, the SSH connection drops, the session ends, or the VM reboots before you repair**, you keep your account and lose the ability to administer or repair the machine. Recovery is then the snapshot, a recovery console, or someone else. That is why Terminal R stays open.

Two practical consequences. Do not start Part C without the `lab3-clean` snapshot actually taken. ⚠ And if you edit `common-auth` instead of `/etc/pam.d/sudo`, **new logins that include it can fail too**, which is a real lockout and not this qualified one.

Exercise 5: A broken chain.

In **Terminal R (root)**, back up the file first:

```
cp /etc/pam.d/sudo /root/sudo.pam.bak
```

Then introduce a fault. On Ubuntu, `/etc/pam.d/sudo` pulls its auth rules in with `@include common-auth`. Open `/etc/pam.d/sudo` in an editor (`nano` is on the image) and replace *that include line* with an `auth` line that **requires a module that does not exist**: the same shape as the `auth` lines in `common-auth` you read in Part B, with the control flag `required` and a module name such as `pam_nonexistent.so`. Change nothing else in the file, and do not touch `common-auth`: that is what keeps *login* working. Read the file back before you test, to confirm your edit is there.

Now, in **Terminal W (student)**, check whether `sudo` still works, with any command that runs something as root (`sudo -k` and `sudo -V` do not count: they never ask PAM).

⚠ **Run `sudo -k` first.** `sudo` remembers a successful authentication for a few minutes, and while it does, it does not call PAM at all: the break stays hidden and `sudo` appears to work. 5a. Record the exact error. What does PAM do when a `required` module cannot be loaded, and why is that the right behaviour for an authentication stack, rather than the opposite?

Repair it, from **Terminal R**:

```
cp /root/sudo.pam.bak /etc/pam.d/sudo
```

Confirm from Terminal W that `sudo id` works again. If anything else is wrong, **restore the `lab3-clean` snapshot**: that is what it is for.

5b. In two or three sentences: what made this failure *safe to experiment with*, and what general precaution does it teach for editing authentication configuration on a machine you care about?

Part D: Hardening the stack

Exercise 6: Account lockout with `pam_faillock`.

`pam_faillock` locks an account after repeated failed authentications. Configure it for `sudo` (again, the safe service to experiment on).

Where the modules live. `pam_faillock.so` and `pam_pwquality.so` are both on the image, but the directory is architecture-specific: `/usr/lib/aarch64-linux-gnu/security/` on an arm64 VM, `/usr/lib/x86_64-linux-gnu/security/` on an amd64 one. Nothing else differs between the two. The stacks read the same on both: `common-auth` uses `[success=1 default=ignore] pam_unix.so nullok`, and `common-password` uses `pam_pwquality.so retry=3`.

`pam_faillock` has to run **before and after** the password check, but in `/etc/pam.d/sudo` the password check (`pam_unix.so`) is hidden inside the `@include common-auth` line. So you cannot add faillock lines around it while that include is there: you replace the include with the auth lines written out in full.

In **Terminal R**, back the file up first:

```
cp /etc/pam.d/sudo /root/sudo.pam.faillock.bak
```

Then read `man 8 pam_faillock`, whose EXAMPLES section shows the shape, and replace the single `@include common-auth` line in `/etc/pam.d/sudo` with an auth stack that:

- checks whether the account is locked **before** asking for the password (`preauth`);
- checks the password with `pam_unix.so`;
- records a failure **after** a wrong password (`authfail`), and clears the count after a right one (`authsucc`);
- denies anything that falls through (`pam_deny.so`), so the stack fails closed;
- uses a low threshold for the demo: `deny=3 unlock_time=120` on every faillock line.

Test it in this order, because a broken stack and a working lock look the same from outside:

1. From **Terminal R**, start from a clean count: `faillock --user student --reset`.
2. From **Terminal W**, run `sudo -k`, then `sudo id` with the **right** password. It must work. If it is refused, your stack is wrong: restore it from Terminal R (`cp /root/sudo.pam.faillock.bak /etc/pam.d/sudo`) and read the man page again.
3. Only then, fail three times: run `sudo -k`, then `sudo id`, and give it a **wrong** password at each of its three prompts. Then read the count:

```
faillock --user student
```

Note that only `sudo` is guarded here: the faillock lines are in `/etc/pam.d/sudo` and nowhere else.

6a. Record the failure count `faillock` shows. What does an attacker attempting online password guessing now face? Can the lockout itself become a threat to the machine and its legitimate users?

When you have answered, put `sudo` back as it was, from **Terminal R**: `cp /root/sudo.pam.faillock.bak /etc/pam.d/sudo`.

Reset the count with `faillock --user student --reset`, from Terminal R if `student` is locked.

Exercise 7: Password quality with `pam_pwquality`.

`pam_pwquality` (installed on the image) enforces password-strength rules in the **password** stack.

```
grep pam_pwquality /etc/pam.d/common-password
```

The rule to install: at least 12 characters, at least one digit and at least one capital letter. As module arguments it reads `minlen=12 dcredit=-1 ucredit=-1`; in `pwquality.conf` each setting goes on its own line (`minlen = 12`, and so on). There are **two ways** to install it, and you choose one:

- in the module's own configuration file, `/etc/security/pwquality.conf`, which applies wherever the module runs; or
- as arguments on the `pam_pwquality.so` line in `/etc/pam.d/common-password`, which apply to that stack only. That file is generated, so a hand edit to it is silently overwritten the next time a package updates the PAM profiles: that is one of the differences between the two.

In **Terminal R**, back up the file you choose (`cp` it to `/root/`), install the rule, and **write down which file and which line you changed**. Restore the file afterwards.

⚠ **If you change nothing, the test still fails, and for the wrong reason.** Ubuntu already rejects passwords shorter than eight characters, so a rejection on its own does not show that your rule is in force. Your record of the file and the line says which rule you installed; the test password you choose is what shows that it, and not the default, refused.

Then, from **Terminal W**, run `passwd`. It asks for your **current** password first, and the rule applies only to the **new** one. Try a new password **chosen to tell the two rules apart**: one Ubuntu's default would accept and your rule should refuse. Observe the rejection, then press Ctrl-C rather than finish the change.

7a. Record the rejection message and the rule that triggered it. Password-quality policy sits in the **password** module type, not **auth**. Explain that placement: what event is it gating, and why is that a different event from a login attempt?

7b. Restore the machine to `lab3-clean` (or confirm all edits are reverted) before you finish.

If it does not work

The one that really stops the lab is the first. Everything else in this lab is recoverable from the shell you already have.

What you see	What to do
You are locked out after editing `common-auth` or the `sudo` stack	You broke the path you would use to repair it, and `student` has no other route to root. Restore the `lab3-clean` snapshot. Prevent it next time by keeping a second root shell open **before** you break anything, and by testing with `sudo -k` in a different terminal rather than closing the one that still works.
After three wrong passwords, `sudo` refuses the **right** one too, with only `Sorry, try again.`	That is the lock working, not a broken stack: it does not say it is a lock. Reset the count from Terminal R (`faillock --user student --reset`) or wait two minutes.

If none of these matches, read the error message in full first: it usually names the layer that refused, and that is most of the answer. Then ask your lab teacher, in the session or by email.

Reflection

Your reflection goes into Lab report 2, as an appendix, beside the reflections from the other labs in this block. We read it for your security reasoning: what you can say about threat, trade-off and defence beyond the exercise itself.

In a short piece (200 to 300 words):

1. Authentication on Linux is *pluggable*: services compose modules rather than hard-coding a password check. What does that architecture buy an administrator, and what new class of mistake does it create (you saw one in Part C)?
2. Part D added two controls, lockout and password quality, that both *reduce* one risk while *introducing* another. Security controls are rarely free. Pick one and describe the trade-off you would explain to the machine's operator.

References

- `pam(8)` and `pam.conf(5)`, https://man7.org/linux/man-pages/man5/pam.conf.5.html
- The Linux-PAM project documentation, https://github.com/linux-pam/linux-pam/tree/master/doc
- `pam_faillock(8)`, https://man7.org/linux/man-pages/man8/pam_faillock.8.html
- `pam_pwquality(8)` and `pwquality.conf(5)`.
- `nsswitch.conf(5)`, https://man7.org/linux/man-pages/man5/nsswitch.conf.5.html
- W. Du, *Computer Security: A Hands-on Approach*, 3rd ed., 2022: the authentication chapters.

Deliverables

There is **no separate graded lab report for this lab**; keep your notes. Block 2's assessed deliverable is **Lab report 2**, produced in Lab 5, and it draws on the authentication understanding you build here. Your **preparation questions P1-P2 are submitted with it** (small weight, but required). Bring your Part B stack-reading notes to Lab 5.